

Roteiro de Atividade: Análise de Dados com Panda

Contexto da Atividade

Você recebeu uma base de dados de vendas de uma empresa e precisa realizar diversas transformações e análises para extrair insights valiosos. A atividade está dividida em missões sequenciais que vão desde a criação de novas colunas até análises agregadas.

Missões de Análise e Transformação

Missão 1: Preparação do Ambiente

O que fazer: Importe as bibliotecas necessárias (pandas e numpy) e carregue o dataset `vendas.xlsx`. Examine a estrutura dos dados verificando as primeiras linhas, informações sobre colunas e valores nulos.

Missão 2: Criar Coluna de Valor da Venda

O que fazer: Crie uma nova coluna chamada `valor_venda` que represente o valor total de cada venda. Esta coluna deve ser o produto da quantidade vendida pelo preço unitário.

Missão 3: Classificar o Nível da Venda

O que fazer:

Crie uma coluna `nivel_venda` que categorize cada venda em:

- Alto: $\text{valor_venda} > 500$
- Médio: $100 \leq \text{valor_venda} \leq 500$
- Baixo: $\text{valor_venda} < 100$

Método 1: Usando `apply` (abordagem funcional)

```
python
# A função apply permite aplicar uma função personalizada a cada elemento de uma
coluna
def classificar_venda(valor):
    if valor > 500:
        return 'Alto'
    elif valor >= 100:
        return 'Médio'
    else:
        return 'Baixo'

# Aplicando a função à coluna 'valor_venda'
df['nivel_venda'] = df['valor_venda'].apply(classificar_venda)
```

Explicação: A função `apply` percorre cada valor da coluna `valor_venda`, aplica a função `classificar_venda` individualmente, e retorna o resultado para a nova coluna `nivel_venda`. É como um loop que processa cada linha separadamente.

Método 2: Usando `loc` com condições (abordagem vetorizada)

```
python
# Método mais eficiente usando loc com condições booleanas
df['nivel_venda'] = 'Baixo' # 1. Define valor padrão para todas as linhas

# 2. Atualiza apenas as linhas que atendem à condição de "Médio"
df.loc[(df['valor_venda'] >= 100) & (df['valor_venda'] <= 500), 'nivel_venda'] =
'Médio'

# 3. Atualiza apenas as linhas que atendem à condição de "Alto"
df.loc[df['valor_venda'] > 500, 'nivel_venda'] = 'Alto'
```

Explicação: Este método trabalha com seleção condicional vetorizada:

1. Primeiro criamos a coluna com um valor padrão ('Baixo') para todas as linhas
2. Usamos `df.loc[condição, coluna] = valor` para modificar APENAS as linhas que atendem a cada condição específica
3. As condições usam operadores booleanos (& para "E", | para "OU")
4. É mais eficiente pois opera em blocos de dados ao invés de linha por linha

Missão 4: Calcular o Imposto

O que fazer: Crie uma coluna `imposto` que calcule o valor do imposto a ser pago com base em duas faixas:

- 15% para vendas acima de R\$ 300
- 10% para vendas até R\$ 300

Use a função `apply` ou método `loc` de maneira similar à missão anterior.

Missão 5: Calcular o Valor Líquido

O que fazer: Crie a coluna `valor_liquido` que representa o valor líquido da venda (valor da venda menos imposto).

Missão 6: Categorizar a Idade dos Clientes

O que fazer: Crie uma coluna `faixa_etaria` que categorize os clientes em três grupos:

- Jovem: menor que 30 anos
- Adulto: entre 30 e 49 anos
- Sênior: 50 anos ou mais

Use a função `apply` ou método `loc` para essa transformação.

Missão 7: Agregações Gerais

O que fazer: Calcule e exiba no terminal as seguintes métricas:

- Total de Vendas Brutas
- Total de Vendas Líquidas
- Total de Impostos
- Número total de vendas
- Número de vendedores únicos
- Número de clientes únicos
- Média do valor da venda
- Ticket médio líquido

Missão 8: Análises por Agrupamento

O que fazer: Realize três análises de agrupamento:

1. Faturamento por Vendedor: Agrupe por vendedor e calcule soma de valor_venda, valor_liquido, imposto e contagem de vendas
 2. Faturamento por Nível da Venda: Agrupe por nível_venda e calcule as mesmas métricas
 3. Faturamento por Faixa Etária: Agrupe por faixa_etaria e calcule as mesmas métricas
-

Missão Final: Exportar para Excel com Múltiplas Planilhas

Objetivo: Salvar os resultados em um arquivo Excel com duas planilhas:

1. Planilha "Dados": Todo o DataFrame processado
2. Planilha "Resumo": Um resumo consolidado das análises

Trecho de código para construir as planilhas:

```
python
# Criar um ExcelWriter para salvar múltiplas planilhas
with pd.ExcelWriter('analise_vendas_final.xlsx', engine='openpyxl') as writer:
```

```

# Planilha 1: Dados completos
df.to_excel(writer, sheet_name='Dados', index=False)

# Planilha 2: Resumo das análises
# Criar um DataFrame de resumo
resumo_data = {
    'Métrica': [
        'Total Vendas Brutas',
        'Total Vendas Líquidas',
        'Total Impostos',
        'Número Total de Vendas',
        'Número de Vendedores Únicos',
        'Número de Clientes Únicos',
        'Ticket Médio Bruto',
        'Ticket Médio Líquido'
    ],
    'Valor': [
        df['valor_venda'].sum(),
        df['valor_liquido'].sum(),
        df['imposto'].sum(),
        df.shape[0],
        df['vendedor'].nunique(),
        df['cliente'].nunique(),
        df['valor_venda'].mean(),
        df['valor_liquido'].mean()
    ]
}

resumo_df = pd.DataFrame(resumo_data)
resumo_df.to_excel(writer, sheet_name='Resumo', index=False)

# Adicionar mais abas com os agrupamentos (opcional)
faturamento_vendedor = df.groupby('vendedor').agg({
    'valor_venda': 'sum',
    'valor_liquido': 'sum',
    'imposto': 'sum',
    'id_venda': 'count'
}).rename(columns={'id_venda': 'Qtd_Vendas'})

faturamento_vendedor.to_excel(writer, sheet_name='Por Vendedor')

print("Arquivo 'analise_vendas_final.xlsx' salvo com sucesso!")
print("Contém 3 planilhas: Dados, Resumo e Por Vendedor")

```

Estrutura do arquivo Excel gerado:

```
text
analise_vendas_final.xlsx
├── Planilha "Dados" (todos os dados processados)
├── Planilha "Resumo" (métricas consolidadas)
└── Planilha "Por Vendedor" (agrupamento por vendedor)
```

Checklist de Conclusão

- Todas as colunas solicitadas foram criadas
 - Os cálculos de impostos e valores líquidos estão corretos
 - As categorizações (nível, faixa etária) foram aplicadas
 - As agregações gerais foram calculadas e exibidas
 - Os agrupamentos solicitados foram realizados
 - O arquivo Excel com múltiplas planilhas foi gerado
-

Dicas para os Alunos:

1. Use `df.head()` frequentemente para verificar cada transformação
2. Teste suas funções `apply` com valores individuais antes de aplicar à coluna inteira
3. Verifique sempre os tipos de dados das colunas antes de operações matemáticas
4. Documente cada passo com comentários no código
5. Explore os DataFrames gerados pelos agrupamentos antes de exportar

Boa análise! 🚀